

Reinforcement Learning

Off-policy Methods with Approximation

Min-hwan Oh

Seoul National University

Episodic Semi-Gradient Sarsa (On-policy)

Episodic Semi-gradient Sarsa for Estimating $\hat{q} \approx q_*$

Input: a differentiable action-value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$

Algorithm parameters: step size $\alpha > 0$, small $\varepsilon > 0$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:

$S, A \leftarrow$ initial state and action of episode (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R, S'

 If S' is terminal:

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$

 Go to next episode

 Choose A' as a function of $\hat{q}(S', \cdot, \mathbf{w})$ (e.g., ε -greedy)

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$

$S \leftarrow S'$

$A \leftarrow A'$

- Can we do off-policy?

- Recall tabular Q-learning

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 until S is terminal

- Recall tabular Q-learning

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 until S is terminal

- Seems like we can parametrize Q with some parameter w as before.

Q-Learning with Function Approximation

- Approximate $Q(s, a)$ with some $Q_{\mathbf{w}}(s, a) = f_{\mathbf{w}}(x(s, a)) \approx Q(s, a)$ where $x(s, a)$ is a feature vector of (s, a)
- Instead of learning the $|S| \times |A|$ dimensional Q-table, the Q-learning algorithm will learn the parameter $\mathbf{w}$
- Consider the loss function: mean-squared error in Q-values

$$\begin{aligned}\ell_{\mathbf{w}}(s, a) &= \mathbb{E}_{s' \sim P(\cdot | s, a)} \left(Q_{\mathbf{w}}(s, a) - R(s, a, s') - \gamma \max_{a'} Q_{\mathbf{w}}(s', a') \right)^2 \\ &=: \mathbb{E}_{s' \sim P(\cdot | s, a)} [\ell_{\mathbf{w}}(s, a, s')]\end{aligned}$$

where

$$\begin{aligned}\ell_{\mathbf{w}}(s, a, s') &= (Q_{\mathbf{w}}(s, a) - \text{target}(s'))^2 \\ \text{target}(s') &= R(s, a, s') + \gamma \max_{a'} Q_{\mathbf{w}}(s', a')\end{aligned}$$

Q-learning with Function Approximation

- Fundamentally, Q-learning with function approximation uses gradient descent to optimize this loss function given sample (target) observations

Algorithm: Q-learning with function approximation

- Take an action a following $\pi(\cdot|s)$
- Observe reward r , transition to $s' \sim P(\cdot|s, a)$
- Update $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t - \alpha_t \nabla_{\mathbf{w}_t} \ell_{\mathbf{w}_t}(s, a, s')$, where

$$\begin{aligned}\nabla_{\mathbf{w}_t} \ell_{\mathbf{w}_t}(s, a, s') &= -\delta_t \nabla_{\mathbf{w}_t} Q_{\mathbf{w}_t}(s, a) \\ \delta_t &= r + \gamma \max_{a'} Q_{\mathbf{w}_t}(s', a') - Q_{\mathbf{w}_t}(s, a)\end{aligned}$$

- $s \leftarrow s'$ and repeat from 1

Deep Q-Network (DQN)

- Use deep neural network to represent Q-function
- $Q_{\mathbf{w}}(s, a) = f_{\mathbf{w}}(x(s, a))$ where $f_{\mathbf{w}}(\cdot)$ is a neural network and $\mathbf{w}$ are weights of the network
- We can use the same approach we discussed earlier

Stability Issues of DQN

Naïve Q-learning oscillates or diverges with neural networks

- Data is sequential and successive samples are correlated
- Policy changes rapidly with slight changes to Q-values
 - ▶ Policy may oscillate
 - ▶ Distribution of data can swing from one extreme to another
- Scale of rewards and Q-values is unknown
 - ▶ Gradients can be unstable when back-propagated

Stable Solutions for DQN

DQN provides a stable solution to deep value-based RL

- Experience replay
- Freeze target Q-network
- Clip rewards to sensible range

Stable Solution 1: Experience Replay

To remove correlations, build dataset from agent's experience

- Take action a_t
- Store sample $(s_t, a_t, r_{t+1}, s_{t+1})$ in replay memory $\mathbf{D}$
- Sample random mini-batch of transitions $\{(s, a, r, s')\}$ from replay memory $\mathbf{D}$
- Optimize MSE. between Q-network and Q-learning target

$$\ell_{\mathbf{w}_k} = \mathbb{E}_{(s,a,r,s')} \left[\left(r + \gamma \max_{a'} Q_{\mathbf{w}_k^-}(s', a') - Q_{\mathbf{w}_k}(s, a) \right)^2 \right]$$

Stable Solution 2: Fixed Target Q-Network

To avoid oscillations, fix parameters used in Q-learning target

- Compute Q-learning target w.r.t old, fixed parameters $\mathbf{w}_k^-$

$$r + \gamma \max_{a'} Q_{\mathbf{w}_k^-}(s', a')$$

Optimize MSE between **Q-learning targets** and Q-network

$$\ell_{\mathbf{w}_k} = \mathbb{E}_{(s,a,r,s')} \left[\left(r + \gamma \max_{a'} Q_{\mathbf{w}_k^-}(s', a') - Q_{\mathbf{w}_k}(s, a) \right)^2 \right]$$

- Periodically update fixed parameters

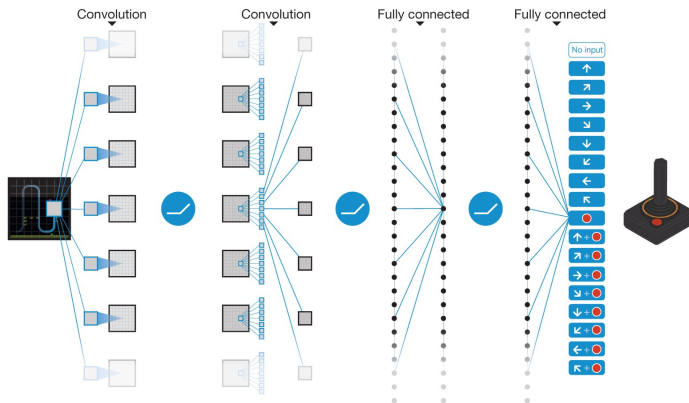
Stable Solution 3: Reward/Value Range

To avoid oscillations, control the reward/value range

- DQN clips the rewards to $[-1, +1]$
- Prevents too large Q-values
- Ensures gradients are well-conditioned

DQN Solving Atari Games

- 49 Atari 2600 games
- Input: $84 \times 84 \times 4$ (4 most recent frames)
- Output: 18 joystick positions



DQN Solving Atari Games

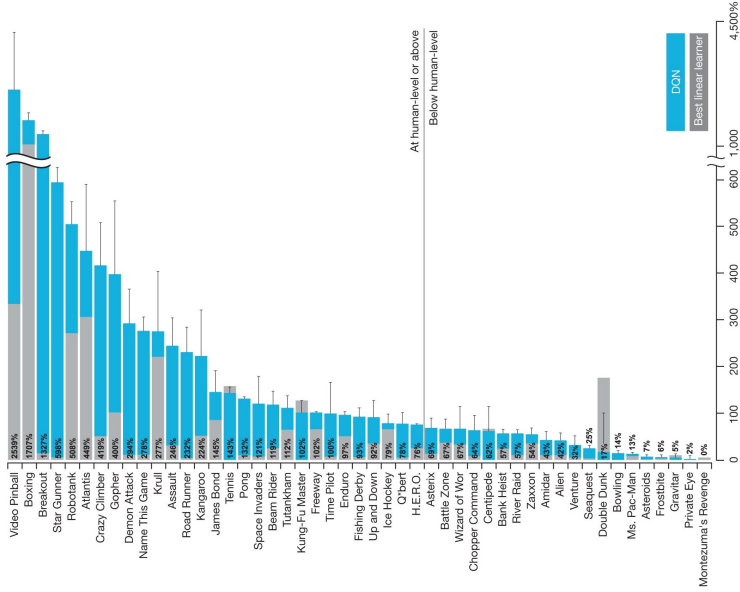
Training

- Use 50 million frames (38 days)
- Replay memory contains 1 million recent frames
- Agent select actions on every 4th frames

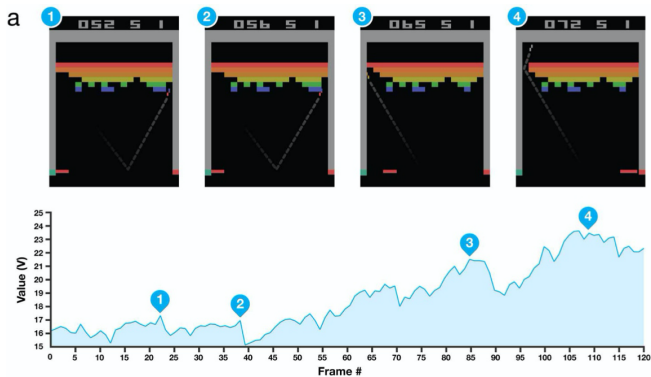
Evaluation

- Agent plays each games 30 times for 5 min with random initial conditions
- Human plays the games in the same scenarios
- Base-line: random agent play in the same scenarios

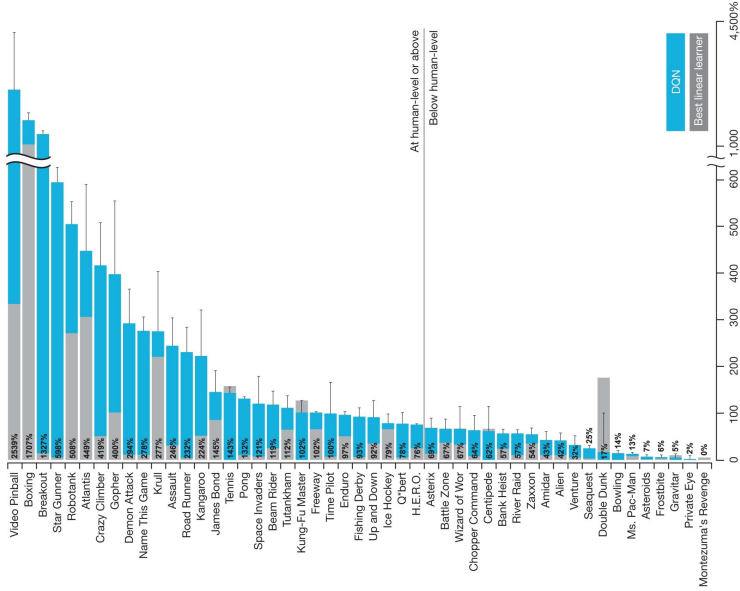
DQN: Atari Performances



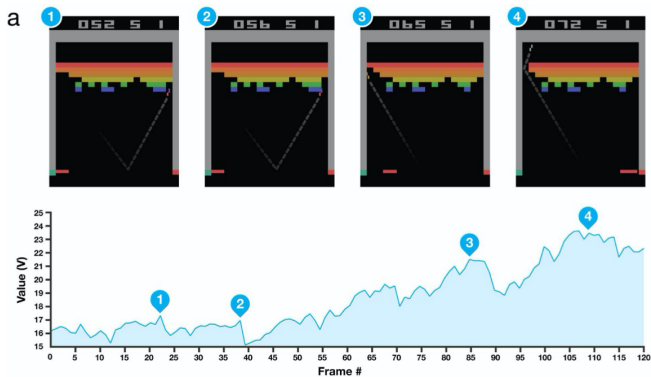
DQN: Atari Performances - Breakout



DQN: Atari Performances



DQN: Atari Performances - Breakout



DQN - summary

- High-level idea: make Q-learning look like supervised learning!
- Engineering tricks to stabilize Q-learning
- DQN agent can learn successful policies directly from high-dimensional

Next class: Chapter 12, Eligibility Traces